MAX PLANCK INSTITUTE
FOR SECURITY AND PRIVACY

SPRING

Still a Long Way to Go

Real-World Steganography Against Content-Based Censorship in Modern Chat Applications

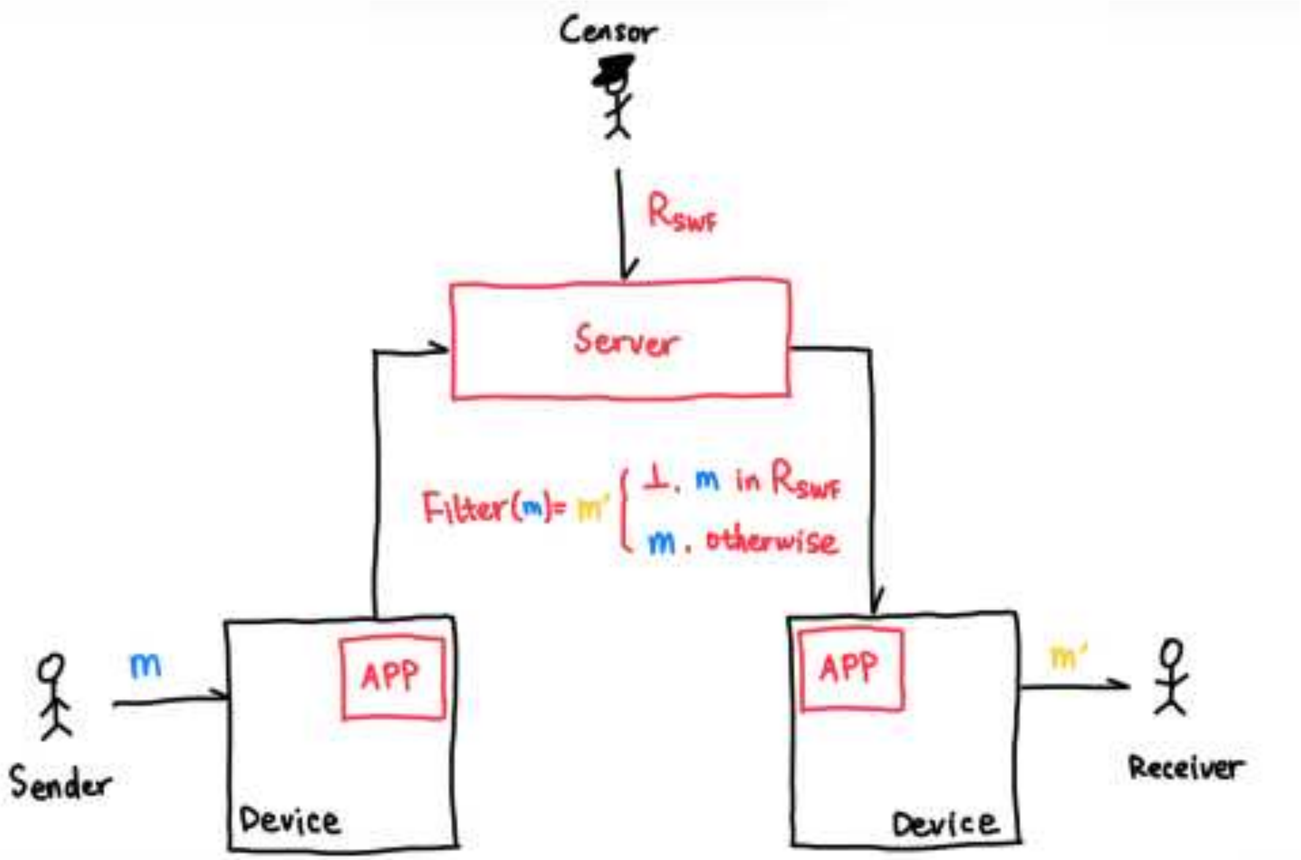
Boya Wang♥, Markus Schiffermüller♣, Klim Kireev♥, Carmela Troncoso♥

SPRING Lab at MPI-SP♥, TU Graz♣

Content-Based Censorship

Sensitive Word Filter (SWF) 敏感词过滤

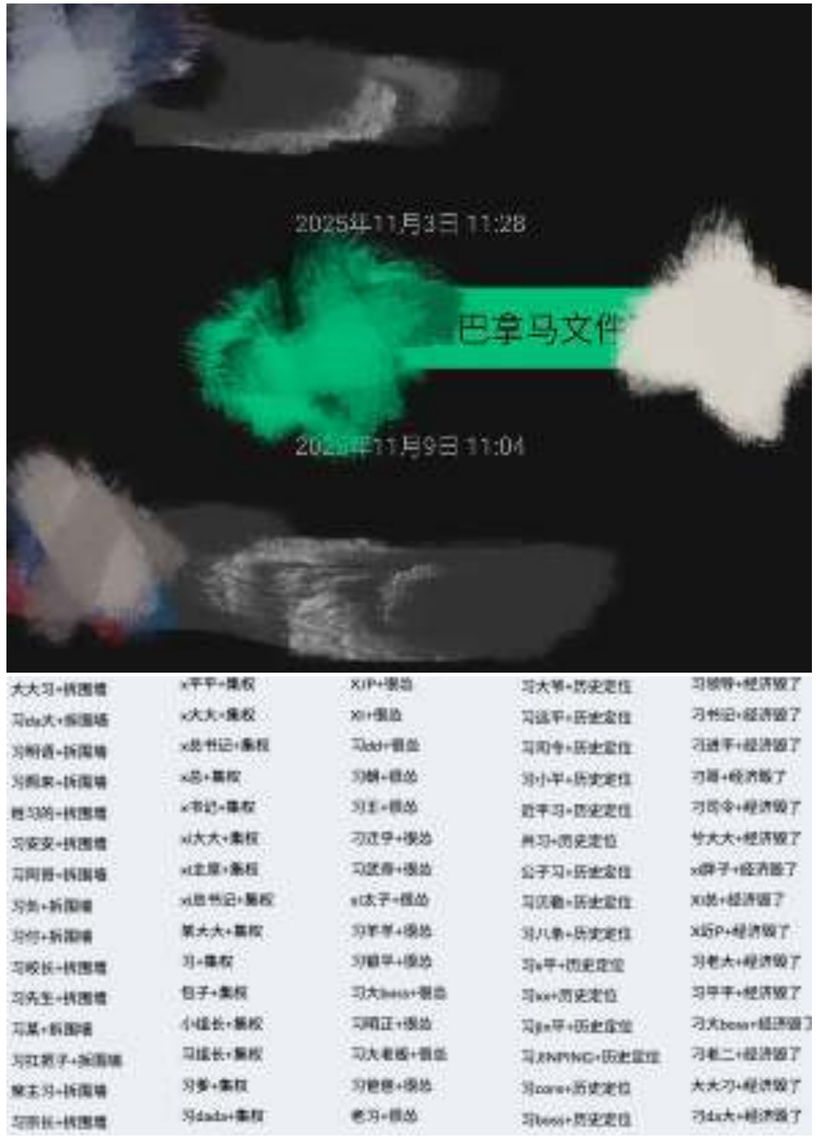
- Match-then-drop contents with any "sensitive word" on non-E2EE Apps



Content-Based Censorship

Sensitive Word Filter (SWF) 敏感词过滤

- Match-then-drop contents with any "sensitive word" on non-E2EE Apps
  - Panama Paper[1] 巴拿马文件
  - xi+centralization[2] 习+集权



[1] A Weixin message containing Panama Paper (巴拿马文件) got dropped silently without notice of the users
[2] A leak of 35,467 censored words about Xi Jinping in 2016, defined by the State Council Information Office https://chinadigitaltimes.net/space/%E5%9B%BD%E6%96%B0%E5%8A%9E

Content-Based Censorship

Sensitive Word Filter (SWF) 敏感词过滤

- Match-then-drop contents with any "sensitive word" on non-E2EE Apps
- Impact on billions of users

Portrait of a SWF Censor

SWF Censor

With nation-state incentives & capabilities

SWF Censor

With nation-state incentives & capabilities

- Adaptive to societal events & dynamics
  - Ad-hoc circumvention can fail in hours

[1] "When the white paper went through the walls around Chinese universities: Speak up, for the possibility of solidarity". Initium Media. SOPA 2023 Awards for Editorial Excellence

SWF Censor

With nation-state incentives & capabilities

- Adaptive to societal events & dynamics
  - Ad-hoc circumvention can fail in hours
  - Addition & removal of rules to minimise collateral damage

SWF Censor

With nation-state incentives & capabilities

- Adaptive to societal events & dynamics
- Combined with censorship on network protocol stack
  - The Great Firewall (GFW) blocking E2EE Apps

SWF Censor

With nation-state incentives & capabilities

- Adaptive to societal events & dynamics
- Combined with censorship on network protocol stack
- Secure circumvention requires undetectable communication
  - Due to legal risk[1,2,3] upon detection of circumvention

[1] Article 6, 1997 Interim Regulations on the Management of International networking of Computer Information
[2] Article 5, 2011 Management Rules for the Security Protection ofInternationally Connected Computer Information Networks
[3] Article 41, 2021 Online Data Security Management Regulations

Towards Circumventing SWF

Against a real-world censor

- Adaptive to societal events & dynamics → systematic design
- Combined with censorship on network protocol stack → “local”circumvention
- Secure circumvention requires undetectable communication → undetectability

Steganographic Circumvention

Steganography
For SWF Censorship Circumvention

- The classic "prisoner's problem"
  - Two prisoners exchange escape plan
  - All communications observed by a guard

Steganography
For SWF Censorship Circumvention

- The classic "prisoner's problem" → SWF circumvention
  - Two prisoners exchange escape plan → Users exchange sensitive information
  - All communications observed by a guard → Via the App controlled by SWF censor

Steganographic Coding Schemes

In previous work from the cryptography community

- Provably secure steganography based on generative models

Steganographic Coding Schemes

In previous work from the cryptography community

- Provably secure steganography based on generative models
  - Assuming a perfect model of the communication channel

Steganographic Coding Schemes

In previous work from the cryptography community

- Provably secure steganography based on generative models
  - Assuming a perfect model of the communication channel
  - Encoding sensitive bits into model output

Steganographic Coding Schemes

In previous work from the cryptography community

- Provably secure steganography based on generative models
  - Assuming a perfect model of the communication channel
  - Encoding sensitive bits into model output
  - Proof of the identical distribution "output **with** embedded sensitive info" vs "output **without** embedded sensitive info"

Steganographic Coding Schemes

In previous work from the cryptography community

- Provably secure steganography based on generative models
  - Assuming a perfect model of the communication channel
  - Encoding sensitive bits into model output
  - Proof of the identical distribution "output **with** embedded sensitive info" vs "output **without** embedded sensitive info"
  - Identical distribution = undetectability = secure circumvention

Steganographic Coding Schemes

In previous work from the cryptography community

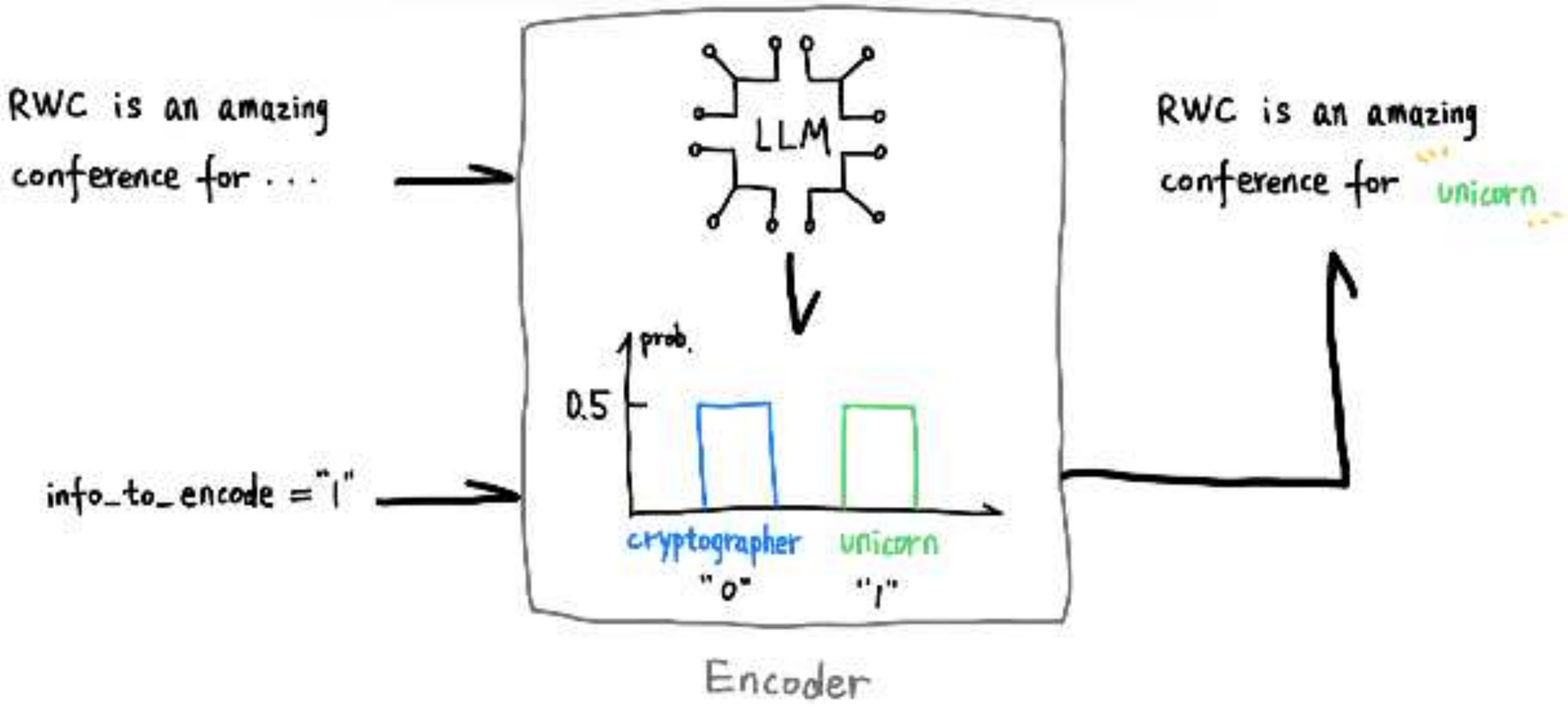
- Using LLMs[1] as the underlying model of human communication

[1] Meteor in RWC'22 talk "Commit Acts of Steganography — Before It's Too Late"

Steganographic Coding Schemes

In previous work from the cryptography community

- Using LLMs[1] as the underlying model of human communication



[1] Meteor in RWC'22 talk "Commit Acts of Steganography — Before It's Too Late"

Deploying Steganography

In Modern Chat Applications

Deploying Steganography

In Modern Chat Applications

- On the lack of contextual considerations in previous work

Deploying Steganography

In Modern Chat Applications

- On the lack of contextual considerations in previous work
  - Entering and exiting of steganography mode

Deploying Steganography

In Modern Chat Applications

- On the lack of contextual considerations in previous work
  - Entering and exiting of steganography mode
  - Interactive chat ≠ LLM output as “a text block”

The Design of Oolong

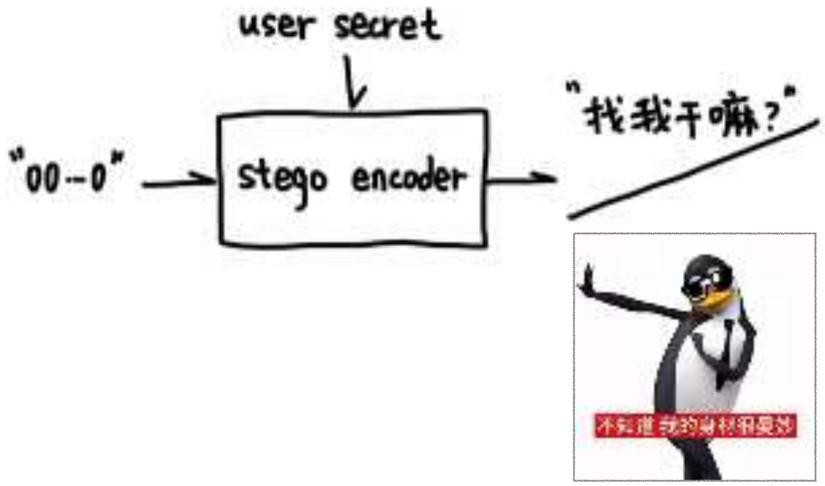
Oolong

Deploying steganography in chat

Oolong

Deploying steganography in chat

- Flexible switching between non-/stego mode
  - Via steganographic “signals”



Oolong

Deploying steganography in chat

- Flexible switching between non-/stego mode
  - Via steganographic “signals”

Oolong

Deploying steganography in chat

- Flexible switching between non-/stego mode
  - Via steganographic “signals”

Oolong

Deploying steganography in chat

- Flexible switching between non-/stego mode
  - Via steganographic “signals”
  - Three phases: idle-init-stego

Oolong

Deploying steganography in chat

- Orchestration of conversation turn-taking

Oolong

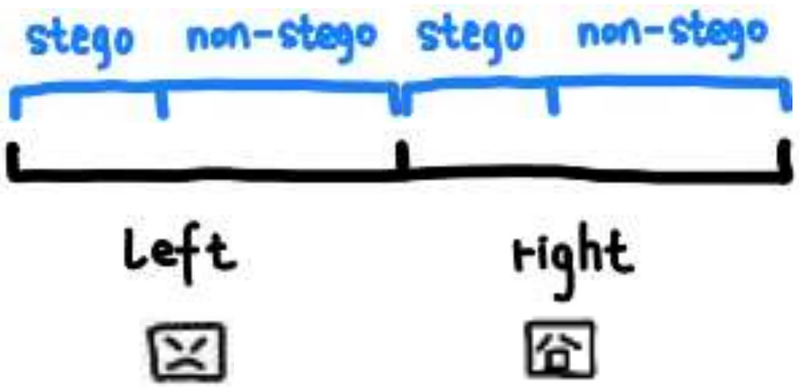
Deploying steganography in chat

- Orchestration of conversation turn-taking
  - “Who sends”

Oolong

Deploying steganography in chat

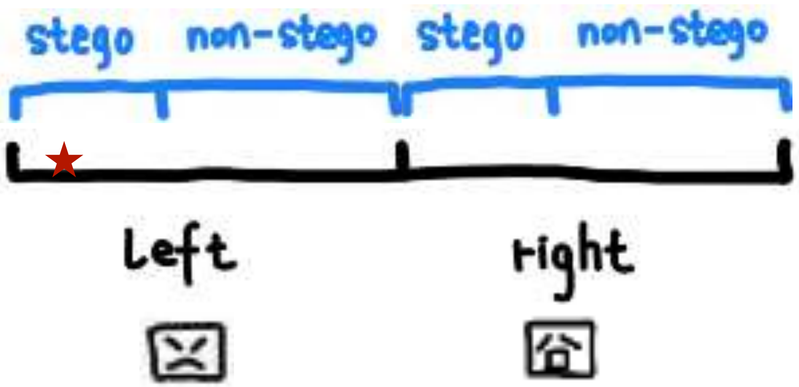
- Orchestration of conversation turn-taking
  - “Who sends”
  - “What kind of message to send”



Oolong

Deploying steganography in chat

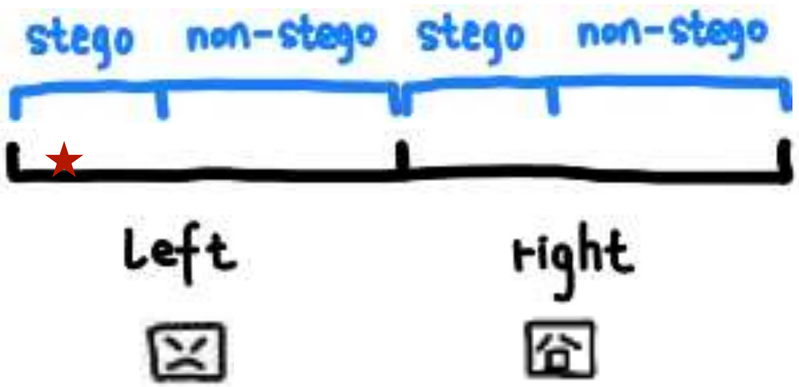
- Orchestration of conversation turn-taking
  - “Who sends”
  - “What kind of message to send”

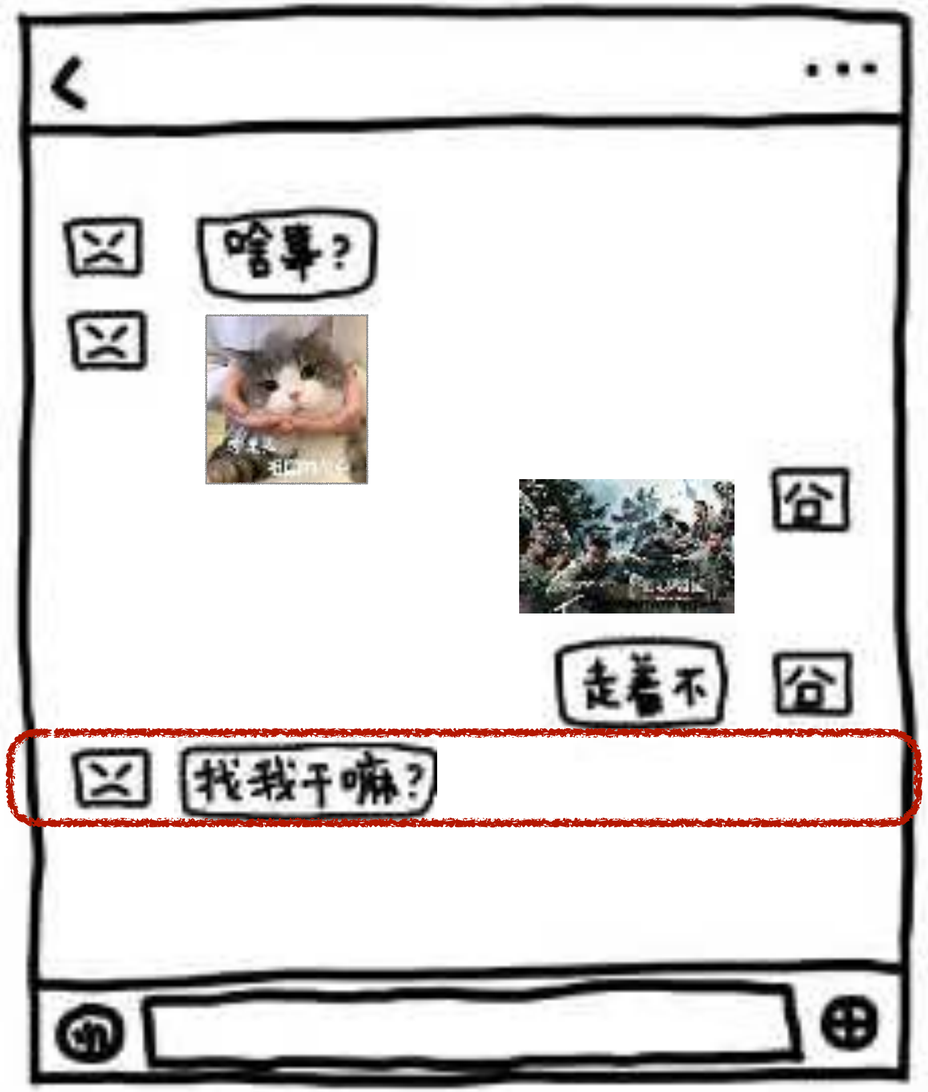


Oolong

Deploying steganography in chat

- Orchestration of conversation turn-taking
  - “Who sends”
  - “What kind of message to send”





Oolong

Protocol overview

Algorithm 1 Orchestrator Main Loop

Require: k_{SR}, signals $\in$ st

```
while true do
    script_SR || m ← GetLatestMsg(script_SR)
    if st.mach is idle then
        if st.siginit in m then
            st.mach ← init
            SPAWN(st, S^stego ← INIT(st, k_SR, script_SR, pp))
        else if startbutton is true then
            SendRoutine(st.siginit)
            startbutton ← false
    else if st.mach is init then
        if st.sigirl, st.sigirr in script_SR || m then
            st.mach ← circ
            SPAWN(st ← CIRCUMVENT(st, S^stego))
    else if st.mach is circ then
        if st.sigidl, st.sigidr in script_SR || m then
            st.mach ← idle
            st ← UPDSIG(st, S^stego)
        else if endbutton is true then
            SendRoutine(st.sigidl/st.sigidr)
            endbutton ← false
```

Algorithm 2 Initialization Function

```
function INIT(st, k_SR, script_SR, pp)
    st.k_coder, st.k_cipht, st.k_sync ← KDF(k_SR, script_SR || 1|.ts)
        ▷ key derivation from pre-shared secret and manuscript
    st.rate, S^stego ← SetCoder(st.k_coder, pp)
                                ▷ steganographic coder setup
    if st.self == MIN(script_SR.side) then
        st.side ← left
        SendRoutine(st.sigcir)
    else                        ▷ differentiate two users based on UID
        st.side ← right
        SendRoutine(st.sigcir)
    return st, S^stego
```

Algorithm 3 Left-User Circumvention Function. There are four probability thresholds that defines the rate limits of which user sends open or stego message: p_{LSO} is the probability of Left-side user Sends an Open message, p_{LSS} is the probability of Left-side user Sends an Stego message; Similarly, p_{RSO} and p_{RSS} are for the right-side user.

```
function CIRCUMVENT(st, S^stego)
    ptbuff, ctbuff ← ∅          ▷ create plain /ciphertext buffer
    p_LSO, p_LSS, p_RSO, p_RSS ← st.rate
        ▷ read four rate limits from coder setup SetCoder
    st.ctr ← 0                  ▷ init stego msg counter
    while True do
        st.ctr ← st.ctr + 1
        r || st.k_cipt ← KDF(st.k_sync, st.ctr)
        r ← NORM(r, [0, 1])
            ▷ pseudorandom selection of user action
        if r ∈ [0, p_LSO) then          ▷ left sending open msg
            m ← FetchRoutine()
            SendRoutine(m)
        else if r ∈ [p_LSO, p_LSO + p_LSS) then
                                        ▷ left sending stego msg
            pt ← FetchRoutine()
            AddToPTBuffer(pt)
            ct ← Preprocess(st.k_cipht, ptbuff)
            e ← Encode(st.k_cipt, ct, S^stego)
            SendRoutine(e)
        else if r ∈ [p_LSO + p_LSS, p_LSO + p_LSS + p_RSO) then
                                        ▷ left receiving open msg
            m ← ReceiveRoutine()
        else if r ∈ [p_LSO + p_LSS + p_RSO, 1) then
                                        ▷ left receiving stego msg
            e ← ReceiveRoutine()
            ct ← Decode(st.k_cipt, e, S^stego)
            AddToCTBuffer(ct)
            pt ← Postprocess(st.k_cipht, ctbuff)
    return st
```

Algorithm 4 Pre-/Postprocessing Function

```
function PREPROCESS(st.k_cipht, ptbuff)
    pt ← Read(ptbuff)           ▷ retrieve newly added plaintext
    pt ← LEN(pt) || pt          ▷ prepending length info
    ptchunk ← Trunc(pt)         ▷ truncating into chunks
    ct ← ENCRYPT(st.k_cipht, ptchunk)   ▷ encryption
    return ct
function POSTPROCESS(st.k_cipht, ctbuff)
    pt ← ∅
    ctchunk ← Read(ctbuff)      ▷ decoded ciphertext
    ptlen ← DECRYPT(st.k_cipht, ctchunk[0])
    if |ctchunk[1 :]| == ptlen then
        ptchunk ← DECRYPT(st.k_cipht, ctchunk[1 :])
        pt ← ptchunk            ▷ updating decryption results
    return pt
```

Oolong

Protocol overview

Algorithm 1 Orchestrator Main Loop

Require: k_{SR}, signals $\in$ st

```
while true do
    script_SR || m ← GetLatestMsg(script_SR)
    if st.mach is idle then
        if st.sigint in m then
            st.mach ← init
            SPAWN(st, S^stego ← INIT(st, k_SR, script_SR, pp))
        else if startbutton is true then
            SendRoutine(st.sigint)
            startbutton ← false
    else if st.mach is init then
        if st.sigirl, st.sigirr in script_SR || m then
            st.mach ← circ
            SPAWN(st ← CIRCUMVENT(st, S^stego))
    else if st.mach is circ then
        if st.sigidl, st.sigidr in script_SR || m then
            st.mach ← idle
            st ← UPDSIG(st, S^stego)
        else if endbutton is true then
            SendRoutine(st.sigidl/st.sigidr)
            endbutton ← false
```

Algorithm 2 Initialization Function

```
function INIT(st, k_SR, script_SR, pp)
    st.k_coder, st.k_cipt, st.k_crypt ← KDF(k_SR, script_SR || 1|.|s)
        ▷ key derivation from pre-shared secret and manuscript
    st.rate, S^stego ← SetCoder(st.k_coder, pp)
        ▷ steganographic coder setup
    if st.self == MIN(script_SR.side) then
        st.side ← left
        SendRoutine(st.sigirl)
    else                    ▷ differentiate two users based on UID
        st.side ← right
        SendRoutine(st.sigirr)
    return st, S^stego
```

Algorithm 3 Left-User Circumvention Function. There are four probability thresholds that defines the rate limits of which user sends open or stego message: p_{LSO} is the probability of Left-side user Sends an Open message, p_{LSS} is the probability of Left-side user Sends an Stego message; Similarly, p_{RSO} and p_{RSS} are for the right-side user.

```
function CIRCUMVENT(st, S^stego)
    ptbuff, ctbuff ← ∅                ▷ create plain /ciphertext buffer
    p_LSO, p_LSS, p_RSO, p_RSS ← st.rate
        ▷ read four rate limits from coder setup SetCoder
    st.ctr ← 0                        ▷ init stego msg counter
    while True do
        st.ctr ← st.ctr + 1
        r || st.k_cipt ← KDF(st.k_cipt, st.ctr)
        r ← NORM(r, [0, 1])
            ▷ pseudorandom selection of user action
        if r ∈ [0, p_LSO) then        ▷ left sending open msg
            m ← FetchRoutine()
            SendRoutine(m)
        else if r ∈ [p_LSO, p_LSO + p_LSS) then
            ▷ left sending stego msg
            pt ← FetchRoutine()
            AddToPTBuffer(pt)
            ct ← Preprocess(st.k_crypt, ptbuff)
            e ← Encode(st.k_cipt, ct, S^stego)
            SendRoutine(e)
        else if r ∈ [p_LSO + p_LSS, p_LSO + p_LSS + p_RSO) then
            ▷ left receiving open msg
            m ← ReceiveRoutine()
        else if r ∈ [p_LSO + p_LSS + p_RSO, 1) then
            ▷ left receiving stego msg
            e ← ReceiveRoutine()
            ct ← Decode(st.k_cipt, e, S^stego)
            AddToCTBuffer(ct)
            pt ← Postprocess(st.k_crypt, ctbuff)
    return st
```

Algorithm 4 Pre-/Postprocessing Function

```
function PREPROCESS(st.k_crypt, ptbuff)
    pt ← Read(ptbuff)                 ▷ retrieve newly added plaintext
    pt ← LEN(pt) || pt                ▷ prepending length info
    ptchunk ← Trunc(pt)               ▷ truncating into chunks
    ct ← ENCRYPT(st.k_crypt, ptchunk) ▷ encryption
    return ct
function POSTPROCESS(st.k_crypt, ctbuff)
    pt ← ∅
    ctchunk ← Read(ctbuff)            ▷ decoded ciphertext
    ptlen ← DECRYPT(st.k_crypt, ctchunk[0])
    if |ctchunk[1 :]| == ptlen then
        ptchunk ← DECRYPT(st.k_crypt, ctchunk[1 :])
        pt ← ptchunk                  ▷ updating decryption results
    return pt
```

Plug in stego schemes from previous work

Empirical Evaluation

Empirical Evaluation

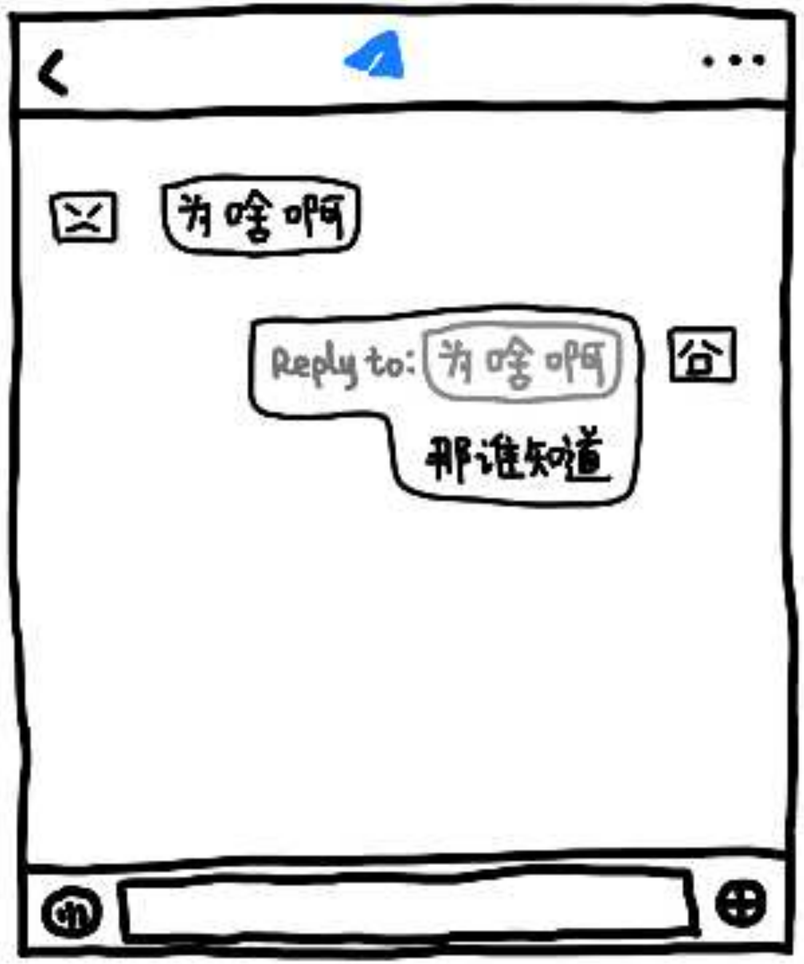
With real-world data & observations of censor

- Dataset of 10k real-world conversations with millions of messages

Empirical Evaluation

With real-world data & observations of censor

- Dataset of 10k real-world conversations with millions of messages
  - Extracting two-user conversations from Telegram public groups*
  - Relying on “reply-to” metadata field



[*] Our data collection, processing, and anonymisation practices were approved by the IRBs

Empirical Evaluation

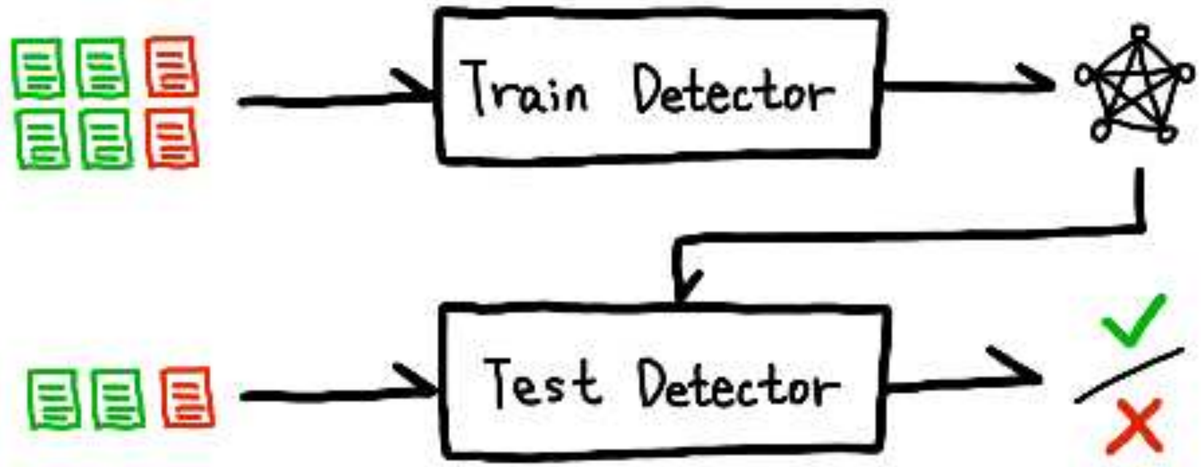
With real-world data & observations of censor

- Simulating censor detection in the worst-case scenario

Empirical Evaluation

With real-world data & observations of censor

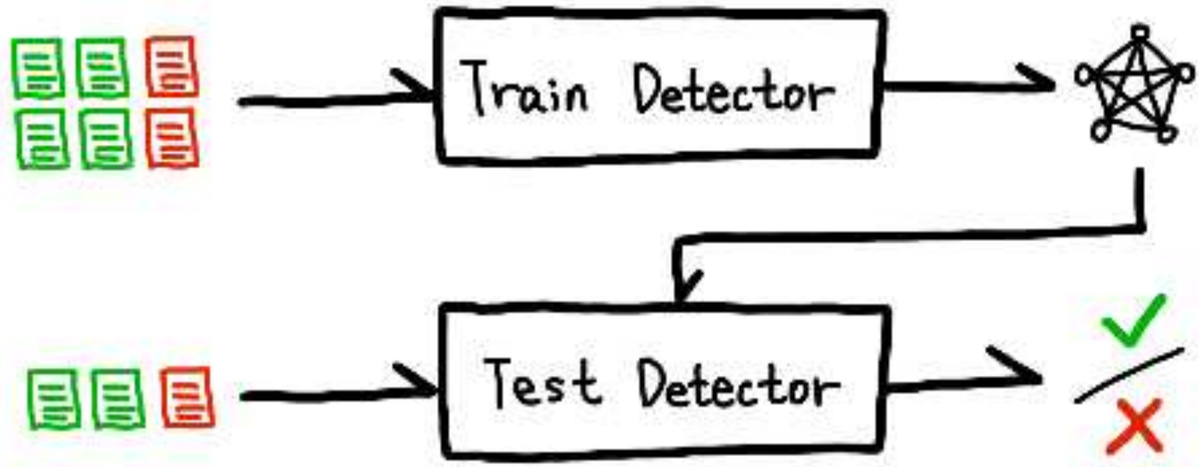
- Simulating censor detection in the worst-case scenario



Empirical Evaluation

With real-world data & observations of censor

- Simulating censor detection in the worst-case scenario
  - Knowledge of Oolong design
  - Persistent access to chat history
  - Correct guess of user parameters



94%

Provably secure LLM stego is empirically detectable with precision

Provable Security $\neq$ Empirical Undetectability

Provable Security $\neq$ Empirical Undetectability

Due to invalid security assumption

- Provable security based on assumption "LLMs speak like human"

Provable Security ≠ Empirical Undetectability

Due to invalid security assumption

- Provable security based on assumption "LLMs speak like human"
- LLMs do not accurately represent chat language

User 1	User 2
Hi	
	How are you?
Fine	
	Nice
…	…
and are the creature	
	that only wants to reach
its full state	
and allow it	
to fight. New	

Provable Security ≠ Empirical Undetectability

Due to invalid security assumption

- Provable security based on assumption "LLMs speak like human"
- LLMs do not accurately represent chat language
  - 5-20x perplexity difference

User 1	User 2
Hi	
	How are you?
Fine	
	Nice
…	…
and are the creature	
	that only wants to reach
its full state	
and allow it	
to fight. New	

Test the security assumptions within their <u>empirical</u> contexts

Lesson Learned I

Feasibility of LLMs in Steganography

Can LLMs become “suitable” for circumvention?

Feasibility of LLMs in Steganography

Can LLMs become “suitable” for circumvention?

- Misaligned optimisation goal of general-purpose LLMs
  - High-quality text vs perfect mimicry of personal chat

Feasibility of LLMs in Steganography

Can LLMs become “suitable” for circumvention?

- Misaligned optimisation goal of general-purpose LLMs
  - High-quality text vs perfect mimicry of personal chat
- Increasingly detectable through time
  - “Garbage in garbage out” effect from context feeding

Feasibility of LLMs in Steganography

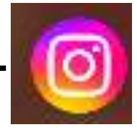
Can LLMs become “suitable” for circumvention?

- Misaligned optimisation goal of general-purpose LLMs
  - High-quality text vs perfect mimicry of personal chat
- Increasingly detectable through time
  - “Garbage in garbage out” effect from context feeding
- Format overfitting of prompt, asymmetric tokenizer, …

What if no LLM?

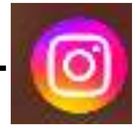
Re-design the Steganographic Coder

For SWF circumvention in chat applications

- Modern chat applications contain social media functionalities
  - ~= + +  +…

Re-design the Steganographic Coder

For SWF circumvention in chat applications

- Modern chat applications contain social media functionalities
  - ~= + +  +…
  - Devices cache large amount of elements, e.g., stickers, links

Re-design the Steganographic Coder

For SWF circumvention in chat applications

- Modern chat applications contain social media functionalities
  - ~=+++…
  - Devices cache large amount of elements, e.g., stickers, links
  - In-app browser provides a universal ranking

Re-design the Steganographic Coder

For SWF circumvention in chat applications

- Modern chat applications contain social media functionalities
- Embedding sensitive info in a dictionary consists of elements
  - Encoding bits into dictionary item indexes (: 0, : 1)
  - Entropy in chat behaviours vs natural language

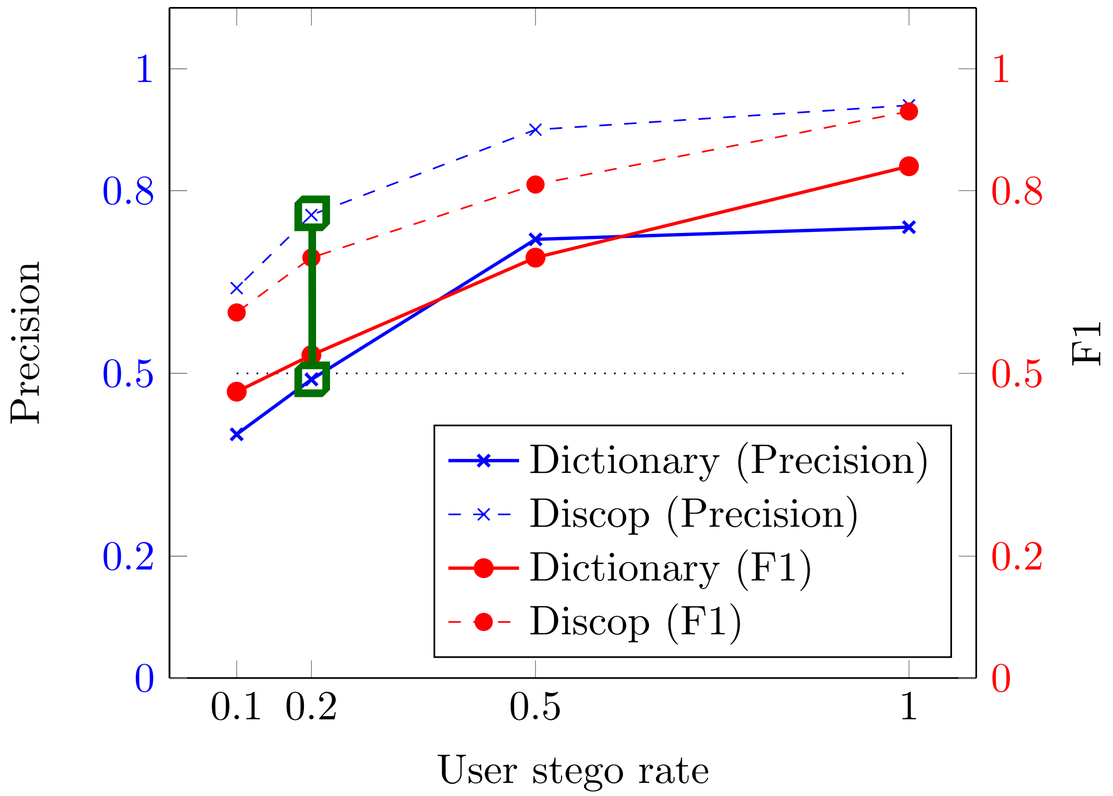
Algorithm 5 Element-Dictionary Instantiation

```
function SETCODER(st.k_coder, pp)
    st.rate, S, st.size, engine ← pp
    S^stego ← ElementFetch(st.k_coder, S, st.size, engine)
                    ▷ storing elements via guided user browsing
    return st.rate, S^stego
function ENCODE(st.k_opt, ct, S^stego)
    dict ← SHUFFLE(st.k_opt, S^stego)
    return e ← dict[ct]
function DECODE(st.k_opt, e, S^stego)
    dict ← SHUFFLE(st.k_opt, S^stego)
    return ct ← INDEX(e, dict)
```

Empirical Evaluation

LLM coder v.s. dictionary coder

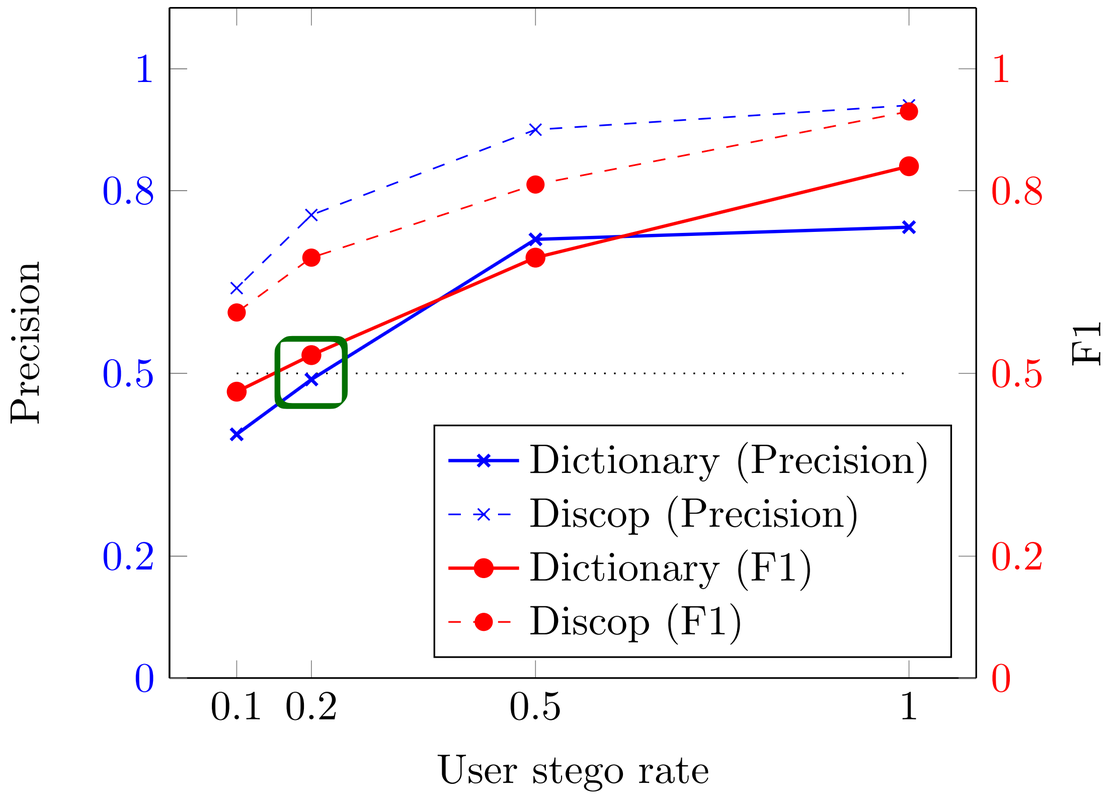
- Dictionary as the computation-wise cheaper & security-wise better solution
  - Detection precision decreases 27%



Empirical Evaluation

LLM coder v.s. dictionary coder

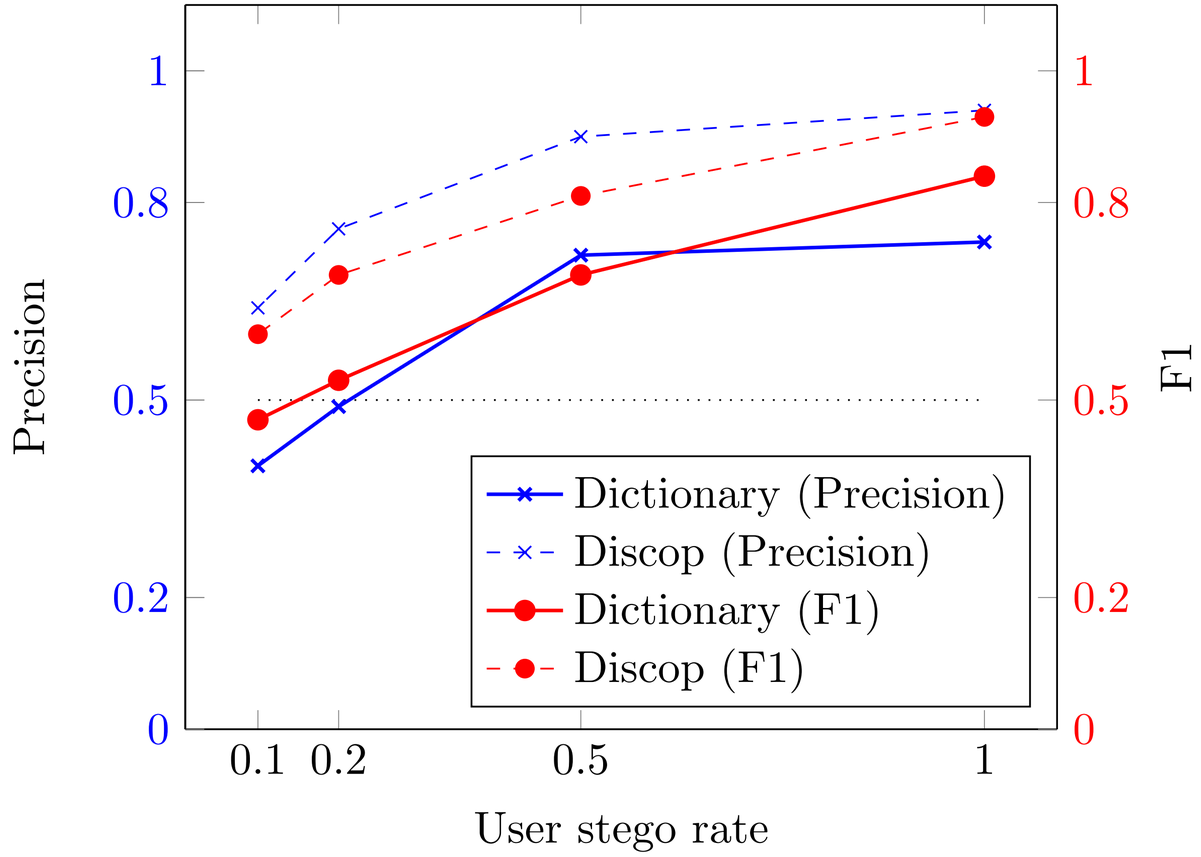
- Dictionary as the computation-wise cheaper & security-wise better solution
  - Detection precision decreases 27%
  - User stego rate = 0.2, a flagged conversation is more likely to be a false positive than a true circumvention



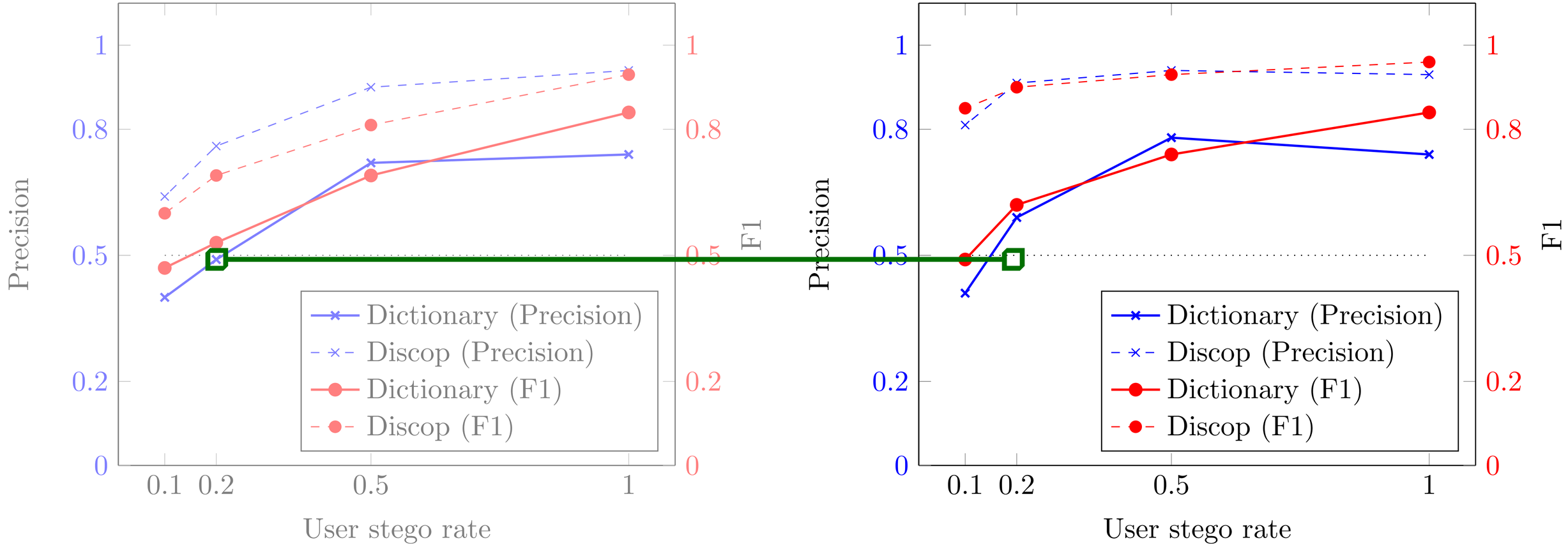
A systematic, <u>contextual</u> design goes a long way

Lesson Learned II

Detection Results on EN



Detection Results on ZH



Empirical Evaluation

Results on Chinese v.s. English

- Dependency on conversation characteristics
  - Element usage pattern difference in EN and ZH conversations

Table 1: Distribution of elements in our conversation datasets

	EN	Ratio	ZH	Ratio
Image	1344	17.5%	2773	30.3%
Link	601	7.8%	1086	11.9%
Sticker	2616	34.0%	1430	15.6%
Video	2408	31.4%	3713	40.6%
Other	712	9.3%	144	1.6%
Total	7681	100%	9146	100%

Test the cryptographic solution in the <u>target</u> environment

Lesson Learned III

Still a Long Way to Go

Real-World Steganography Against Content-Based Censorship in Modern Chat Applications

- Do empirical tests on theoretical results to examine the underlying assumptions
- Do take the context into consideration when designing a cryptographic solution
- Do evaluate the solutions in the target environment we aim to deploy

Still a Long Way to Go

Real-World Steganography Against Content-Based Censorship in Modern Chat Applications

- Do empirical tests on theoretical results to examine the underlying assumptions
- Do take the context into consideration when designing a cryptographic solution
- Do evaluate the solutions in the target environment we aim to deploy

革命尚未成功，同志仍須努力 —— 《總理遺屬》

“the revolution has not succeeded, there is <u>still a long way to go</u>”

Still a Long Way to Go

Real-World Steganography Against Content-Based Censorship in Modern Chat Applications

- Do empirical tests on theoretical results to examine the underlying assumptions
- Do take the context into consideration when designing a cryptographic solution
- Do evaluate the solutions in the target environment we aim to deploy

Boya Wang♥(boya.wang@mpi-sp.org), Markus Schiffermüller♣, Klim Kireev♥, Carmela Troncoso♥

SPRING Lab at MPI-SP♥, TU Graz♣